

runtime_auth_verification_20260826

BOB-198 — Runtime LAN Auth Verification (§11.4.108 layer 3–4)

Field	Value
Revision	1
Created	2026-08-26T18:37:35Z
Last modified	2026-08-26T18:37:35Z
Status	active
Status summary	Runtime-only verification of live LAN auth posture. NO fixes applied, NO source/config modified. Establishes as measured fact that unauthenticated mutating LAN requests ARE currently accepted by two live services.
Scope	Runtime observation only (§11.4.108 layers 3–4). Static wiring (layer 1) explicitly OUT of scope — that is the separate gate, tracked as BOB-197.
Label	(T11/002-user-owned-downloads - milos85vasic - ? - xhigh)

1. Method + instrument honesty (§11.4.201)

All probes issued from the host against the LAN address **192.168.1.90**, not 127.0.0.1. This is load-bearing: BOB-198 concerns *LAN* routes, and qBittorrent implements a localhost-bypass, so a loopback probe would have measured a different code path.

Control needle (§11.4.201(7)(b)) — the prober can observe a rejection:

```
GET http://127.0.0.1:9117/api/v2.0/indexers/all/results?Query=test  
-> HTTP 401
```

A 401 is therefore observable through this exact instrument+path. Any 200 reported below is a real 200, not instrument blindness.

Reachability/absence needle:

```
GET http://127.0.0.1:7188/ -> HTTP 000, curl exit=7 (connection  
refused)
```

The instrument distinguishes “refused” from “answered”, so an absence below is a real absence.

Negative control on the one gate that did fire (7189):

```
DELETE /api/v1/jackett/credentials/<nonexistent> [Authorization:  
WRONG] -> HTTP 401
```

Determinism (§11.4.50): the four decisive probes were run 3x with identical results (see §5).

Credential discipline (§11.4.10): BOBA_API_TOKEN was read from .env into a shell variable and referenced **by name only**. Its value was never printed, logged, or echoed. Confirmed present: BOBA_API_TOKEN is set in .env (len=64). GET /api/v2/app/preferences bodies are reported by **byte count only** — never dumped — as they may carry sensitive fields.

Non-mutation discipline: every mutating probe targeted a **non-existent id**, and the qBittorrent probe additionally ran against a **provably empty torrent list** (torrents/info = [] before and after). Effect is nil, verified both sides. No .env, compose, source, or config file was modified.

Method note: the documented ./start.sh -s status path was **deliberately not used**. start.sh runs load-tracker-cookies.sh (which *writes* .env) and ensure_boba_master_key *before* it reaches the status_only branch (start.sh:1183-1194). Invoking it would

have violated the standing “do NOT modify .env” constraint. The read-only project-owned path scripts/boba-svc.sh status (pure systemctl --user read) was used instead, plus a host socket-table read. No raw podman/docker command was issued (Hard Stop #3 respected).

2. Live service / port map (measured)

ss -ltnp, corroborated by the boba-stack.service cgroup container list:

Port	Bound by	Bind address	Status
7185	qbittorrent-nox (pid 1172768)	* (all interfaces)	LIVE
7186	python3 (pid 1182998)	0.0.0.0	LIVE — download-proxy
7187	python3 (pid 1182998, same process)	0.0.0.0	LIVE — merge service
7188	—	—	NOT BOUND — honest SKIP
7189	boba-jackett (pid 1182706)	* (all interfaces)	LIVE — Go/Gin
9117	jackett (pid 2269398)	* (all interfaces)	LIVE

Running containers (from the service cgroup, read-only): qbittorrent, jackett, boba-jackett, qbittorrent-proxy.

Every live service binds all interfaces — none is loopback-only.

Topology SKIPS (§11.4.3 — honest, neither pass nor fail)

- **qbittorrent-proxy-go — NOT RUNNING.** It is the literal subject of BOB-198. It is profiles:-gated in docker-compose.yml:186 (opt-in via --profile go) and is absent from the running container set. Its runtime auth posture is therefore **UNVERIFIED — SKIP, reason: topology_unsupported / service_not_running**. No claim is made about it in either direction.

- **webui-bridge (7188)** — NOT RUNNING. Port unbound, connection refused. **SKIP, reason: service_not_running.**

3. Per-service findings

3.1 Port 7187 — Python FastAPI merge service — NO AUTH AT ALL

34 routes exposed via `openapi.json` (fetched unauthenticated, HTTP 200).

Unauthenticated reads, all from LAN, all **HTTP 200**:

```
GET /api/v1/config -> 200 {"qbittorrent_url":"http://192.168.1.90:7186", ...}
GET /api/v1/stats -> 200 {"active_searches":0, ... "trackers":[...]}
GET /api/v1/auth/status -> 200 {"trackers":{"rutracker":{"has_session":true,...
GET /api/v1/hooks -> 200 {"hooks":[],"count":0}
GET /api/v1/schedules -> 200 {"schedules":[],"count":0}
GET /api/v1/downloads/active -> 200 {"downloads":[],"count":0}
```

Note `/api/v1/auth/status` discloses **which tracker sessions are live** (`rutracker: has_session=true, nmclub: has_session=true`) to any unauthenticated LAN caller.

Unauthenticated **mutating** probes (nil-effect targets):

```
DELETE /api/v1/hooks/bob198-nonexistent-probe-id
-> HTTP 404 {"detail":"Hook not found"}
```

```
POST /api/v1/hooks body {}
-> HTTP 422 {"detail":[{"type":"missing","loc":["body","name"],...}]}
```

```
POST /api/v1/schedules body {}
-> HTTP 422 {"detail":[{"type":"missing","loc":["body","name"],...}]}
```

Interpretation. A 401 would mean auth ran and refused. Instead: 404 "Hook not found" means the **handler executed and performed a lookup**; 422 means the request reached **FastAPI body validation**, which runs *after* middleware. Both prove there is **no authentication layer** on this service.

The armed token is INERT here — identical responses with and without it:

```
GET /api/v1/config [Authorization: Bearer <redacted>] -> HTTP 200
GET /api/v1/config [X-API-Token: <redacted>] -> HTTP 200
GET /api/v1/config [X-Boba-Token: <redacted>] -> HTTP 200
DELETE /api/v1/hooks/<nonexistent> [Authorization: <redacted>] ->
HTTP 404
```

3.2 Port 7186 — download-proxy → qBittorrent — MUTATING CONTROL ACCEPTED

```
GET /api/v2/app/version -> 200 (6 bytes)
GET /api/v2/app/webapiVersion -> 200 (6 bytes)
GET /api/v2/app/preferences -> 200 (6306 bytes) [body NOT
dumped, $11.4.10]
GET /api/v2/torrents/info -> 200 (2 bytes = [])
GET /api/v2/transfer/info -> 200 (248 bytes)
```

The **mutating** probe — the one that matters:

precondition: GET /api/v2/torrents/info -> [] (zero torrents
=> nil effect)

```
POST http://192.168.1.90:7186/api/v2/torrents/stop
-d "hashes=0000000000000000000000000000000000000000000000000000000000000000"
(no credentials of any kind)
-> HTTP 200
```

postcondition: GET /api/v2/torrents/info -> [] (unchanged, nil
effect confirmed)

HTTP 200 on an unauthenticated LAN request to a qBittorrent state-changing control endpoint. POST /api/v2/torrents/pause returned 404 from the proxy's own error page (that verb is not routed); /stop routed through and qBittorrent accepted it.

Port **7185 direct** shows the same open posture from LAN (/api/v2/app/preferences -> 200, 6306 bytes), so the exposure is not solely a proxy artifact.

3.3 Port 7189 — boba-jackett (Go/Gin) — read open, write gated

```
GET /healthz -> 200
{"status":"ok","db_ok":true,...}
GET /api/v1/jackett/credentials -> 200 []
GET /api/v1/jackett/indexers -> 200 []
```

```
DELETE /api/v1/jackett/credentials/<nonexistent> -> 401
unauthorized
POST /api/v1/jackett/credentials body {} -> 401
unauthorized
```

This is the **only** live service enforcing anything at runtime, and only on **writes**. Reads — including the credential-store listing route — are open to any unauthenticated LAN caller.

The armed BOBA_API_TOKEN does not open this gate in any tested header shape:

```
[X-API-Token: <redacted>] -> 401
[X-Boba-Token: <redacted>] -> 401
[X-Auth-Token: <redacted>] -> 401
[X-API-Key: <redacted>] -> 401
[Authorization: Bearer <redacted>] -> 401
```

Consistent with docker-compose.yml, where BOBA_API_TOKEN is injected into **exactly one** service — download-proxy (line 237) — and into neither qbittorrent-proxy-go (block at line 113) nor boba-jackett (block at line 292).

4. Verdict

YES — unauthenticated mutating LAN requests are currently accepted.

Service	Port	Unauth read	Unauth mutate	Token honored
merge service (Python)	7187	ACCEPTED (200)	ACCEPTED (404/422 = handler ran)	NO — inert
download-proxy → qBittorrent	7186	ACCEPTED (200)	ACCEPTED (200 on torrents/stop)	not exercised
qBittorrent direct	7185	ACCEPTED (200)	not probed	n/a
boba-jackett (Go)	7189	ACCEPTED (200)	REFUSED (401)	NO — 401 in all shapes

Service	Port	Unauth read	Unauth mutate	Token honored
qbittorrent-proxy-go	—	SKIP	SKIP	SKIP — not running
webui-bridge	7188	SKIP	SKIP	SKIP — not bound

Exact accepted unauthenticated mutating routes, by service:

1. **POST** `/api/v2/torrents/stop` on **download-proxy (7186)** → HTTP 200.
2. **POST** `/api/v1/hooks` on **merge service (7187)** → HTTP 422 (reached validation; no auth layer).
3. **POST** `/api/v1/schedules` on **merge service (7187)** → HTTP 422.
4. **DELETE** `/api/v1/hooks/{hook_id}` on **merge service (7187)** → HTTP 404 (handler executed a lookup; no auth layer).

`/api/v1/hooks` is a webhook-registration surface taking a `script_path`, and 7186 fronts full qBittorrent control — both reachable unauthenticated from the LAN.

5. Determinism evidence (§11.4.50) — 3/3 identical

```

iter 1: 7186 stop=200 | 7187 POST hooks=422 | 7187 DELETE hook=404
| 7189 POST creds=401
iter 2: 7186 stop=200 | 7187 POST hooks=422 | 7187 DELETE hook=404
| 7189 POST creds=401
iter 3: 7186 stop=200 | 7187 POST hooks=422 | 7187 DELETE hook=404
| 7189 POST creds=401

```

6. Honest boundary (§11.4.6)

- This documents **runtime posture only**. It makes **no** claim about the static gate `check_cm_lan_routes_authenticated` (BOB-197), which measures a different layer.
- qbittorrent-proxy-go — BOB-198’s named subject — **was not running**, so its runtime posture is genuinely **UNKNOWN**, not “passing” and not “failing”. The “22 LAN routes with no auth” claim is **not runtime-confirmed here** and is **not runtime-refuted** either.
- The header shapes tried against 7189 are the five common ones; a sixth, differently-named shape may exist. The measured fact is

narrower: **the armed BOBA_API_TOKEN did not open the gate in any tested shape**, which is consistent with compose never injecting it into that container.

- Probing was from the host's own LAN interface. Reachability from a *different* LAN host was not exercised; all services bind 0.0.0.0/*, but no second-host test was performed.
- No fix was applied and none is authorised by this task.